Reflection Types and Provability Logic in OCaml

Pratik Deshmukh
192.192 Project in Logic and Artificial Intelligence 1
2025W
MSc Logic and Computation, TU Wien

1. Background

1.1 Reflection Types in OCaml

Reflection types enable programs to inspect and manipulate their own type information at compile-time or runtime. In OCaml, this capability is primarily achieved through **Generalized Algebraic Data Types (GADTs) [3]**, which extend the type system by allowing constructors to constrain type parameters differently. This turns the type checker into a theorem prover that verifies program properties at compile-time.

A key concept is **type witnesses**, values that carry proofs of type equality:

```
type (_, _) type_eq = Refl : ('a, 'a) type_eq
```

This witness proves that two types are equal, enabling safe type-level computation and casting.

The type ('a, 'a) type_eq is only inhabited when both type parameters are identical. When pattern matching on Refl, the type checker learns that the two types are equal, enabling type-safe casts without runtime checks.

1.2 Provability Logic (GL)

Gödel-Löb provability logic (GL) is a modal logic that formalizes reasoning about provability in arithmetic. It extends propositional logic with a modal operator □ (box) representing "provably" and is characterized by two key axioms:

- **Distribution axiom (K):** □(p → q) → (□p → □q)
- **Löb's axiom (GL):** □(□p → p) → □p

Here p and q range over all GL formulas (including nested modalities). These are axiom schemas, any substitution instance is an axiom. For example, □(□p → p) → □p applies even when p itself contains □ operators.

GL captures the properties of formal provability predicates and connects to fundamental results in mathematical logic, including Gödel's incompleteness theorems.

1.3 Connection Between Reflection Types and Provability Logic

Both domains share deep theoretical connections:

1. **Type witnesses ↔ Equality proofs:** GADTs encode limited dependent types within ML-family type systems (e.g., OCaml, Haskell), where type witnesses correspond to equality proofs in type theory.
2. **Pattern matching ↔ Proof by cases:** Pattern matching on GADTs mirrors proof by cases in theorem provers, with the type system ensuring exhaustiveness.
3. **LCF approach (named after Milner's Logic for Computable Functions):** HOL Light, implemented in OCaml, uses abstract data types to represent theorems, ensuring soundness through the type system, similar to how GADTs enforce proof validity. [1]

2. Problem Statement

2.1 Research Questions

This project investigates:

1. **How can OCaml's reflection capabilities be used to implement a type-safe proof checker for GL?**
2. **What practical benefits do GADTs provide for implementing modal logic proofs?**
3. **How does this approach differ from existing mechanizations of GL in proof assistants like HOL Light?**

2.2 Existing Work

The most relevant existing work is the mechanization of GL in HOL Light by Maggesi and Perini Brogi (2023). Their work makes the following contributions:

- It formalizes GL's metatheory in HOL Light
- It proves soundness and completeness w.r.t. possible world semantics, and
- It implements a theorem prover for GL as a HOL Light tactic

Key difference to the present work: Their work mechanizes GL *within* a theorem prover (HOL Light), focusing on formal verification of the logic itself. This project explores using OCaml's *reflection types* to implement a type-safe GL proof checker, emphasizing the type system's capabilities rather than formalization in a proof assistant.

3. Solution: Implementation

The project consists of three interconnected components demonstrating reflection types in practice.

3.1 Type-Safe GL Proof Checker

Location: `editor/src/gl_proof_checker.ml`

3.1.1 Formula Representation with Phantom Types

```
type _ formula =
  | False : [`Prop] formula
  | True : [`Prop] formula
  | Atom : string -> [`Prop] formula
  | Not : 'a formula -> 'a formula
  | And : 'a formula * 'a formula -> 'a formula
  | Or : 'a formula * 'a formula -> 'a formula
  | Impl : 'a formula * 'a formula -> 'a formula
  | Box : [`Prop] formula -> [`Modal] formula
  | BoxProp : [`Prop] formula -> [`Prop] formula
```

Phantom types `[`Prop]` and `[`Modal]` distinguish propositional formulas from modal formulas at the type level, preventing invalid formula constructions.

3.1.2 Deep Embedding of Proofs as GADTs

```
type _ gl_proof =
  | AxiomK : 'a formula * 'a formula -> 'a formula gl_proof
  | AxiomS : 'a formula * 'a formula * 'a formula -> 'a formula gl_proof
  | AxiomLob : [`Prop] formula -> [`Prop] formula gl_proof
  | AxiomBoxK : [`Prop] formula * [`Prop] formula -> [`Prop] formula gl_proof
  | ModusPonens : 'a formula gl_proof * 'a formula gl_proof -> 'a formula gl_proof
  | Necessitation : [`Prop] formula gl_proof -> [`Modal] formula gl_proof
```

Each proof constructor encodes inference rules at the type level. For example, `Necessitation` can only accept a proof of a propositional formula and produces a proof of a modal formula.

Note on modal nesting: The type system distinguishes propositional formulas ([Prop]) from modal formulas ([Modal]). Nested box operators like □□p are supported via the BoxProp constructor, which keeps formulas at the propositional level. The Necessitation rule accepts only propositional proofs, reflecting standard GL proof theory where necessitation applies to theorems. This design ensures type safety while supporting the full expressiveness of GL.

3.1.3 Proof Checking

```
let rec check_proof : type a. a gl_proof -> validity = function
  | AxiomK _ | AxiomS _ | AxiomLob _ | AxiomBoxK _ -> Valid
  | ModusPonens (pf1, pf2) ->
      (* Verify that pf1 proves (p → q) and pf2 proves p *)
  | Necessitation pf -> check_proof pf
```

The proof checker validates inference rules while the type system ensures structural correctness.

3.1.4 Kripke Semantics

```
module Semantics = struct
  type frame = {
    worlds : world list;
```

```
    relation : (world * world) list;
    valuation : string -> world -> bool;
  }

  let rec eval_formula : [`Prop] formula -> frame -> world -> bool
  let validate_proof : [`Prop] formula gl_proof -> frame -> bool
end
```

Implements semantic validation using Kripke frames. A Kripke frame consists of a set of possible worlds and an accessibility relation between them. For GL (provability logic), frames must be transitive (if w_1 accesses w_2 and w_2 accesses w_3, then w_1 accesses w_3) and irreflexive (no world accesses itself). These constraints correspond to the properties of the provability predicate in Peano Arithmetic. The implementation checks that GL axioms, particularly Löb's axiom, hold in all such frames.

3.1.5 Reflection: Compile-Time to Runtime

```
type 'a formula_repr =
  | RFalse : [`Prop] formula_repr
  | RTrue : [`Prop] formula_repr
  | RAtom : string -> [`Prop] formula_repr
  (* ... *)

let rec reify_formula : type a. a formula -> a formula_repr
```

Demonstrates reflection by converting compile-time type-indexed formulas to runtime representations that can be inspected and manipulated. The reify_formula function traverses a formula whose type is statically known (e.g., [Prop] formula) and produces a corresponding runtime value (formula_repr`) that preserves the type information. This enables runtime operations like pretty-printing, comparison, or serialization while maintaining the type safety guarantees established at compile time.

3.2 Practical GADT Examples

Location: `editor/src/practical_gadts.ml`

Implements four practical patterns demonstrating GADT applications:

3.2.1 Type-Safe AST for Functional Language

```
type _ term =
  | Int : int -> int term
  | Bool : bool -> bool term
  | Lambda : string * 'a term -> ('b -> 'a) term
  | Plus : int term * int term -> int term
  | If : bool term * 'a term * 'a term -> 'a term

let rec eval : type a. (string * value) list -> a term -> value
```

Type-indexed AST ensures type safety during evaluation, e.g. attempting `Plus (Bool true, Int 5)` produces a compile-time type error.

3.2.2 Editor Commands with Phantom Types

```
type saved = Saved
type modified = Modified

type ('saved, 'cursor) editor_state = {
```

```
  content: string list;
  saved_state: 'saved;
  cursor_state: 'cursor;
}

type ('s1, 'c1, 's2, 'c2) command =
  | Insert : string -> (saved, cursor_valid, modified, cursor_valid) command
  | Save : (modified, 'c, saved, 'c) command
```

Phantom types track editor state at the type level, preventing invalid operations (e.g., saving an already-saved file).

3.2.3 Type-Safe Syntax Highlighting

```
type_token =
  | TKeyword : string -> string token
  | TNumber : int -> int token
  | TString : string -> string token

let get_color : type a. a token -> color = function
  | TKeyword _ -> Blue
  | TNumber _ -> Green
  | TString _ -> Yellow
```

Guarantees that each token type has a consistent color, verified at compile-time.

3.3 Editor Integration

Location: `editor/src/`

A syntax-aware editor demonstrating practical GADT applications:

- **File Mode Detection** (`editor.ml`): Automatically detects `.gl` files for GL proof mode
- **GL Syntax Checking** (`gl_syntax.ml`): Validates GL syntax and provides feedback
- **Proof Execution** (Ctrl-E): Runs GL proof demonstrations interactively
- **Brace Matching** (`editor_gadts.ml`): Type-safe matching of parentheses, braces, and brackets
- **Command History**: Type-safe undo/redo using reversible commands

3.4 Supporting Components

Functional Language Interpreter (Python)

Location: `interpreter/`

Implements a functional language with:

- Lambda calculus with currying

- Lazy `{}` vs eager `[]` records
- Record-as-environment pattern
- Built-in arithmetic operations

Purpose: Demonstrates language design principles that motivate type-safe implementations in the editor's AST module.

4. Technical Contributions

4.1 Type Safety Guarantees

The GADT-based approach provides compile-time guarantees that traditional implementations cannot:

1. **Invalid proofs cannot be constructed:** The type system rejects ill-formed proof terms
2. **Modal/propositional distinction enforced:** Cannot mix modal and propositional formulas incorrectly
3. **Inference rules structurally correct:** Pattern matching ensures exhaustive case coverage

4.2 Comparison with HOL Light Mechanization

Aspect	Maggesi & Perini Brogi (HOL Light)	This Project (OCaml GADTs)
Focus	Formalize GL metatheory	Type-safe proof checker
Approach	Mechanization in proof assistant	Reflection types in programming language
Soundness	Proved within HOL Light	Guaranteed by OCaml type system
Proof Terms	HOL Light theorems	OCaml GADT values
Execution	Tactic-based proof search	Direct proof construction

4.3 Reflection Demonstration

The project demonstrates three levels of reflection:

1. **Compile-time reflection:** GADTs enable type-level computation
2. **Runtime reflection:** `reify_formula` converts types to inspectable values
3. **Cross-level bridge:** Type witnesses connect compile-time and runtime representations

---

5. Conclusion

This project demonstrates how OCaml's reflection types, particularly GADTs, can implement a type-safe proof checker for Gödel-Löb provability logic. Unlike existing mechanizations that formalize GL within proof assistants, this work explores the type system's capabilities for ensuring proof correctness at compile-time.

The implementation provides:

A complete GL proof checker with type-level guarantees

Practical GADT patterns applicable to real-world programming

Integration demonstrating reflection types in a working editor

Connections between type theory and modal logic

The work connects to broader themes in logic and computation: the LCF approach to theorem proving, the Curry-Howard correspondence between proofs and programs, and the role of type systems in program verification.

References

---

1. Maggesi, M., & Perini Brogi, C. (2023). Mechanising Gödel-Löb provability logic in HOL Light. *Journal of Automated Reasoning*.
2. Boolos, G. (1995). *The Logic of Provability*. Cambridge University Press.
3. Yallop, J., & White, L. (2014). Lightweight higher-kinded polymorphism. *Functional and Logic Programming*.
4. Project Repository: https://github.com/inquisitour/reflection-types
5. Comprehensive research document: https://github.com/inquisitour/reflection-types/docs/Research.md